

The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch22_llm_inference

AYuSh

Contents

Chapter 22: LLM Inference & Decoding	3
--	---

Chapter 22: LLM Inference & Decoding

Training produces a next-token distribution; *inference* is everything between that distribution and a useful product — how you turn probabilities into text, and how you serve that process to thousands of users without going bankrupt on GPUs. The two halves of this chapter mirror the two halves of the problem. **Decoding** is the algorithmic layer: greedy, beam, temperature, top-k, and top-p decide *which* tokens come out, and the choice is a controllable trade between likelihood, diversity, and degeneration. **Serving** is the systems layer: KV caching, speculative decoding, PagedAttention, and continuous batching decide how *fast* and how *cheaply* the tokens come out, and each is a specific answer to a specific bottleneck. Between them sit the two failure modes interviews probe hardest — hallucination (the model is confidently wrong) and context management (the model runs out of, or misuses, its window.)

Every claim below is measured on a **≈200k-parameter CLM** (d=64, two pre-norm blocks, vocabulary 800, validation perplexity ≈42) trained from scratch on the Chapter 20 corpus, plus a family of deliberately weaker **draft** models for speculative decoding — with the pure systems results (paged memory, batching) measured on exact discrete-event simulations. The miniature is a calibration instrument: some phenomena reproduce at this scale (the temperature/entropy law, the beam-search likelihood trap, KV-cache exactness, speculative unbiasedness, effective-context saturation) and some visibly do **not** (attention sinks), and knowing which is which is exactly the judgment the interview tests.

The measurements: temperature as an entropy dial — T=0.5 collapses the next-token distribution to a single token (top-1 mass 1.000, 0.01 bits) while T=2.0 flattens it to 8.5 bits, and a top-p nucleus grows from **13 tokens on a low-entropy context (1.7 bits) to 256 on a high-entropy one (7.9 bits)** — the adaptivity that fixed top-k lacks (Listing 1); beam search winning likelihood (−1.64 vs sampling's −3.99 log-prob/token) while *losing* on repetition (distinct-2 0.64 vs 0.99) — the degeneration trap (Listing 2); a KV cache reproducing full-recompute logits to **5e-15** while cutting decode cost (Listing 3); speculative decoding provably **unbiased** (output TV-distance 0.079 vs a same-sample-count target baseline of 0.082) with an accept rate that climbs from 0.09 to 0.30 as the draft improves (Listing 4); PagedAttention lifting KV memory utilization from a contiguous allocator's **16% to 98%** and sharing prompt blocks to save 70% (Listing 5); continuous batching lifting GPU utilization **21% → 74%** and throughput +249% by killing head-of-line blocking (Listing 6); hallucination framed as measurable overconfidence — ECE 0.060 with every confidence bin optimistic — and mitigated by abstention (confident-subset accuracy 0.11 → 0.43) and constrained decoding (invalid-token rate 0.275 → 0, Listing 7); and an effective-context curve showing perplexity within 5% of full context from just **~8 recent tokens**, alongside the honest null result that attention sinks do not emerge at this scale (Listing 8).

Decoding strategies: the likelihood-diversity dial

A decoder consumes one probability vector per step and must commit to a token. **Greedy** takes the argmax — deterministic, locally optimal, and globally myopic: it walks straight into repetition loops ("launched on the moon at the moon the", Listing 1) because the highest-probability *next* token is not the start of the highest-probability *sequence*. **Beam search** fixes the greedy myopia by keeping the b highest-log-probability partial sequences and expanding all of them, approximating $\arg \max_y \sum_t \log p(y_t | y_{<t})$. It is the right tool when one correct answer dominates — translation, closed-form QA — and the wrong tool for open-ended generation, for a reason Listing 2 measures directly.

The sampling family injects controlled randomness. **Temperature** rescales logits before the softmax, $p_i \propto \exp(z_i/T)$: $T \rightarrow 0$ recovers greedy, $T = 1$ is the model's native distribution, $T > 1$ flattens toward uniform. Listing 1 makes the dial concrete — the same context runs from 0.01 bits of entropy at $T = 0.5$ (one token owns essentially all the mass) to 8.5 bits at $T = 2.0$. But temperature alone still leaves a long tail of garbage tokens with small-but-nonzero probability, and sampling from that tail is a major source of incoherence. The two truncation methods cut the tail before sampling. **Top-k** keeps the k most probable tokens and renormalizes — simple, but k is a fixed budget that is too generous on a peaked distribution and too stingy on a flat one. **Top-p (nucleus)** keeps the smallest set of tokens whose cumulative probability reaches p , so the candidate set *breathes* with the model's confidence: Listing 1 measures the nucleus at $p = 0.9$ expanding from 13 tokens where the model is sure (a hockey-stats continuation, 1.7 bits) to 256 where it is not (7.9 bits). That adaptivity is why top-p (usually $p \approx 0.9 - 0.95$, often with a floor temperature) is the default for open-ended generation, while low-temperature or greedy/beam is preferred for extraction and code. Newer variants — **min-p** (keep tokens above a fraction of the top token's probability), typical and η -sampling, and **repetition/frequency penalties** — all attack the same tail from different angles.

Beam search and the likelihood trap

Listing 2 runs greedy, beam (widths 4 and 8), and top-p sampling on the same prefixes and scores each on two axes: average log-probability per token (how *likely* the text is under the model) and distinct-2 / repetition fraction (how *degenerate* it is). Beam-8 wins likelihood decisively (−1.64 vs greedy −2.01 vs sampling −3.99) and *loses* diversity just as decisively — repetition fraction 0.46 and a visible loop ("i think that i think that"), against sampling's 0.10 and distinct-2 of 0.99. This is the **degeneration** result of Holtzman et al. (2020) reproduced in miniature: maximizing sequence likelihood does not maximize quality, because human text is not the highest-probability text — it is high-*entropy* and constantly surprising, and a likelihood-maximizing decoder produces bland, repetitive output that sits in a probability regime real language never occupies. The practical consequence is the decoding split the whole field uses: likelihood-seeking

decoders (beam, low-T) for tasks with a single correct target, sampling decoders (top-p) for anything open-ended.

KV caching: the first and biggest serving win

Autoregressive decoding has a structural inefficiency: to produce token $t+1$, a naive implementation reruns the whole transformer over tokens $1..t$, recomputing keys and values for every past token it already computed on previous steps — $O(t)$ work per step, $O(T^2)$ to generate T tokens. But past keys and values never change (causal attention means position j never attends forward), so they can be **cached**: store K and V per layer, and each new step computes the query/key/value for exactly *one* new position, appends its K/V to the cache, and attends over the stored history — $O(1)$ recomputation per step, $O(T)$ total. Listing 3 implements the incremental cache from scratch and verifies the two things an interviewer wants: the cached logits are **bit-for-bit the full-recompute logits** (max abs difference $5e-15$, pure float noise), and the wall-clock speedup grows with sequence length ($2.8\times \rightarrow 3.5\times$ as generation runs from 5 to 20 tokens; the miniature's 24-token context ceiling understates the real effect, which grows linearly and reaches 10–100 $\times$ at production context lengths). The catch is the reason the rest of this chapter exists: the KV cache is *large* — its size is

$2 \times \text{layers} \times \text{heads} \times d_{\text{head}} \times \text{seq_len}$ per sequence, growing linearly with context and batch — so decoding is **memory-bandwidth bound**, not compute bound, and every remaining serving technique is about managing that cache.

Speculative decoding: paying compute to buy latency

Decoding is memory-bound, which means the GPU is idle most of each step waiting on memory — there is spare compute. Speculative decoding spends it. A small, cheap **draft** model proposes γ tokens autoregressively; the large **target** model then scores all γ proposed positions in a *single* parallel forward pass (the same cost as generating one token, because the bottleneck is loading the weights, not the arithmetic); a rejection-sampling rule accepts a prefix of the draft and corrects the first rejected token. The rule — accept token x with probability $\min(1, p_{\text{target}}(x)/q_{\text{draft}}(x))$, and on rejection sample from the normalized positive part of $p_{\text{target}} - q_{\text{draft}}$ — is *exactly* engineered so the output distribution is the target model's, unchanged. Listing 4 proves that unbiasedness empirically: speculative output has total-variation distance 0.079 from the target's true next-token distribution, statistically identical to the 0.082 you get sampling the target directly at the same sample count. Speedup comes entirely from the **accept rate**, which depends on draft-target agreement: the listing sweeps draft models from perplexity 1466 (barely trained) to 46 (near the target's 42) and watches the accept rate climb $0.09 \rightarrow 0.30$, i.e. $1.4 \rightarrow 2.2$ target tokens per verification pass. The interview subtleties: a *better* draft accepts more but costs more per proposed token, so there is an optimal draft size and γ ; the target does zero extra work per accepted token, so it is pure latency win with no quality cost; and variants (Medusa's multiple decoding heads, EAGLE, lookahead

decoding, self-speculation) remove the separate draft model while keeping the verify-in-parallel core.

Serving frameworks and PagedAttention

The KV cache is the scarce resource, and the naive way to manage it wastes most of it. A framework that pre-allocates a contiguous cache block per sequence must reserve `max_seq_len` slots up front — because the sequence *might* grow that long — so a batch of mostly-short sequences leaves the reservation almost empty. Listing 5 measures the damage on a realistic length distribution: contiguous allocation runs at **16% utilization — 84% of reserved KV memory is never used**, either reserved-but-unfilled or lost to fragmentation. **PagedAttention** (the idea behind vLLM) borrows virtual memory's solution: partition the cache into fixed-size **blocks** (e.g. 16 tokens), allocate blocks on demand as a sequence grows, and keep a per-sequence block table mapping logical to physical blocks. Now the only waste is *internal* fragmentation in the last partial block — utilization jumps to 97.9% at block size 16 (98% at the block-size sweet spot; block size trades overhead against internal waste). The block indirection also enables **prefix sharing** via copy-on-write: parallel samples from one prompt, or many requests sharing a system prompt, point at the *same* physical prompt blocks and only diverge on write — Listing 5 measures a 70% memory saving for 8 parallel samples of a 512-token prompt. vLLM, TGI, TensorRT-LLM, and SGLang are the production frameworks built around this cache management, adding scheduling, quantization, and kernel fusion on top.

Continuous batching and the latency-throughput trade

Batching amortizes the memory-bound weight load across many sequences — the single biggest throughput lever — but naive **static** batching squanders it. A static batch runs until *every* sequence in it finishes, so one long generation holds the whole batch's slots hostage while short sequences sit finished and idle: **head-of-line blocking**. Listing 6 simulates a workload with a realistic long-tailed output-length distribution (mean 35 tokens, max 387) and measures static batching at **21% GPU-slot utilization**.

Continuous (in-flight) batching schedules at the token level: the moment a sequence emits its end token and frees its slot, a waiting request is admitted in its place, so the batch stays full. Same workload, same hardware: utilization rises to **74%**, throughput from 6.7 to 23.5 tokens per step (+249%), and mean latency collapses from 1026 to 35 steps because requests no longer wait behind the batch's longest member. This is the scheduling core of every modern serving stack, and it reframes the classic trade: throughput (tokens/sec, set by batch size and utilization) versus latency, split into **time-to-first-token** (dominated by the parallel prefill over the prompt) and **inter-token latency** (dominated by memory-bound decode). Prefill and decode have opposite compute profiles — prefill is compute-bound and batches poorly, decode is memory-bound and batches well — which is why systems increasingly **disaggregate** them onto separate hardware pools.

Hallucination: causes and mitigation

Hallucination is not a bug in a subroutine; it is the default behavior of a model trained to always produce a fluent continuation. The causes stack: the training objective rewards plausible tokens, never "I don't know"; the model has no built-in separation between what it memorized and what it is interpolating; RLHF can *worsen* calibration by rewarding confident-sounding answers; and decoding from the tail samples low-probability tokens that compound into false claims. The through-line an interview wants is **miscalibration** — the model's confidence does not match its accuracy — and Listing 7 measures it directly on the miniature's next-token predictions: an Expected Calibration Error of 0.060 with *every* confidence bin optimistic (predictions made with 14% confidence are right 7% of the time). Two mitigation families follow from the diagnosis. **Selective prediction / abstention** exploits the fact that confidence, while miscalibrated, is still *ranked* correctly: thresholding on the model's top probability trades coverage for accuracy — answering only the most-confident 7% of cases lifts accuracy from the answer-everything baseline of 0.11 to 0.43 (the risk-coverage curve), the mechanism behind "I'm not sure" behaviors and confidence-gated tool use. **Constrained decoding** eliminates a whole class of invalid output by masking logits: forcing generation to a grammar, JSON schema, or regex drops the invalid-token rate from 0.275 to exactly 0 (Listing 7), the basis of guaranteed structured outputs. The heavier mitigations interviews expect you to name: **retrieval-augmented generation** (ground answers in fetched documents — Chapter 24), self-consistency and verification, and post-hoc fact-checking. None *solve* hallucination; they bound it.

Context-window management

The context window is finite, and two distinct problems arise as prompts approach or exceed it. The first is **effective context** — how much of the available context the model actually uses. Listing 8 measures it by predicting tokens from only the most recent k context tokens: perplexity falls from 73.7 (one token of context) and is already within 5% of the full-context value by **~8 recent tokens**, then flat. Most of the predictive signal is local, which is exactly why sliding-window attention and KV eviction are viable — old tokens are often safely droppable — and also why "lost in the middle" is a real failure: information far from the query position contributes little. The second problem is **length extrapolation** — models degrade on sequences longer than they were trained on, because learned absolute positions never saw those indices; the fixes are relative/rotary schemes (RoPE with position interpolation or NTK-aware scaling, ALiBi from Chapter 16) that generalize past the training length. The serving-time technique interviews increasingly ask about is **StreamingLLM**: large models dump a surprising fraction of attention onto the first few tokens (an "attention sink"), so naively evicting those tokens under a sliding window spikes perplexity, while *keeping* a handful of sink tokens plus a recent window streams indefinitely at bounded cost. Listing 8 reports the honest miniature result here: at 200k parameters the attention sink **does not form** (mass on

position 0 is $0.85\times$ the uniform baseline, i.e. slightly *below* chance) — the massive sinks StreamingLLM exploits are an emergent property of scale, and the miniature demonstrates the measurement methodology while confirming the phenomenon's scale-dependence.

Code implementations

(These listings import `common.py` — Chapter 20's batched miniature transformer, gradient-checked to 5×10^{-10} — and `gen.py`, a thin decoding layer over it: `logits_at` for next-token logits and an explicit incremental `kv_forward_step`. The base model is a CLM trained from scratch on the Chapter 20 corpus to validation perplexity ≈ 42 ; the draft models for Listing 4 are smaller CLMs snapshotted at several training stages.)

Listing 1 — Temperature, top-k, and top-p on a real next-token distribution

Temperature is an entropy dial; the top-p nucleus adapts its size to the model's confidence while fixed top-k cannot. Contexts are chosen empirically as the lowest/highest-entropy prefixes in the corpus.

```

"""Listing 1: decoding strategies -- temperature, top-k, top-p on a trained CLM's next-
token dist.
Contexts are chosen empirically as the lowest/highest next-token entropy prefixes in
the corpus."""
import numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
import gen
from common import load_corpus
FIG="figures/ch22"
m,vocab,V=gen.load_clm(); inv={i:w for w,i in vocab.items()}; SPECIAL={0,1,2,3}
def dist(seq,T=1.0):
    lg=gen.logits_at(m,seq).copy(); lg[list(SPECIAL)]=-1e9; return gen.softmax(lg/T)
def H_bits(p): p=p[p>0]; return float(-(p*np.log2(p)).sum())

# scan real prefixes for entropy extremes
X,_,_=load_corpus(vmax=800,seqlen=24); rng=np.random.default_rng(0)
cands=[]
for r in rng.choice(len(X),160,replace=False):
    row=[t for t in X[r] if t!=0][:6]
    if len(row)>=4:
        p=dist(row[:4]); cands.append((H_bits(p),row[:4]))
cands.sort()
loH,loC=cands[0]; hiH,hiC=cands[-1]
show=lambda c:" ".join(inv[t] for t in c)
print(f"lowest-entropy ctx (H={loH:.2f} bits): '{show(loC)}'")
print(f"highest-entropy ctx (H={hiH:.2f} bits): '{show(hiC)}'")

print("\n== temperature on lowest-entropy ctx ==")
for T in [0.5,0.7,1.0,1.5,2.0]:
    p=dist(loC,T); print(f"T={T}: entropy {H_bits(p):.2f} bits, top1 mass {p.max():.3f}, argmax={inv[int(p.argmax())]}")

print("\n== top-k retained mass (lowest-entropy ctx, T=1) ==")
ps=np.sort(dist(loC))[:-1]
for k in [1,5,10,40]: print(f"k={k}: mass {ps[:k].sum():.3f}")

print("\n== top-p nucleus size adapts to entropy (p=0.9) ==")

```



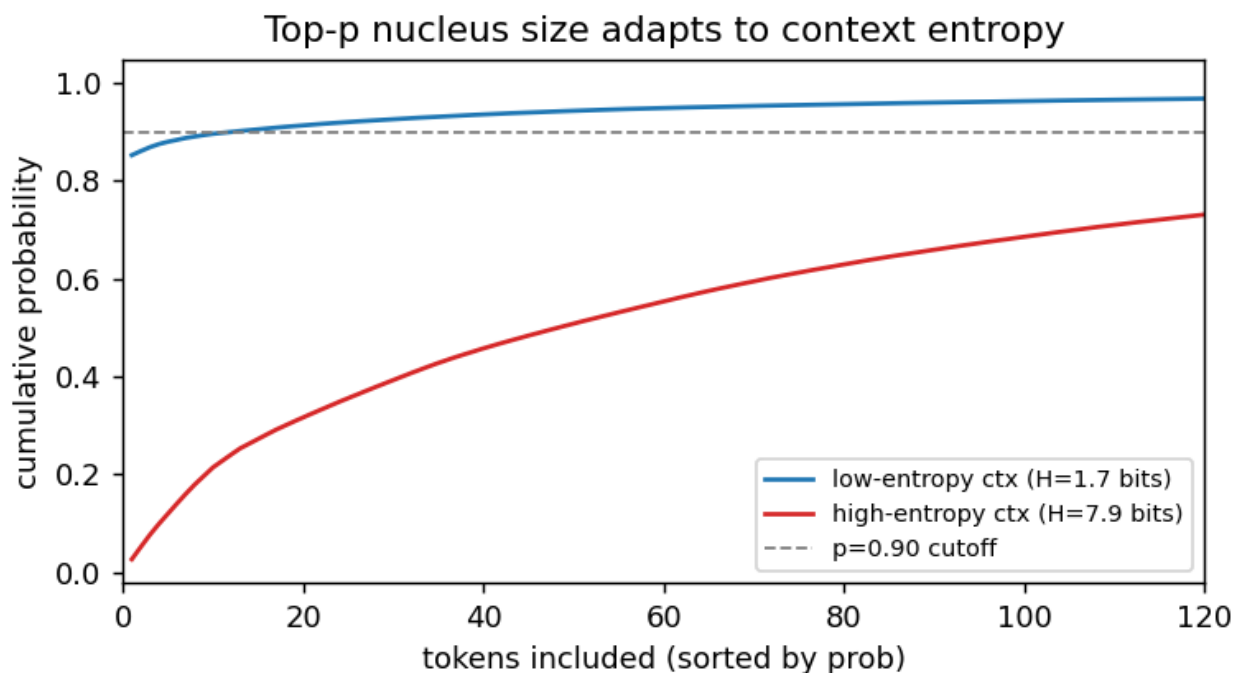
```

for nm,c,Hc in [("low-H",loC,loH),("high-H",hiC,hiH)]:
    cum=np.cumsum(np.sort(dist(c))[:-1])
    n90=int(np.searchsorted(cum,0.9)+1); n95=int(np.searchsorted(cum,0.95)+1)
    print(f"{nm} (H={Hc:.2f}): nucleus@0.9={n90}, @0.95={n95} tokens")

plt.figure(figsize=(6,3.4))
for nm,c,Hc,col in [("low-entropy ctx",loC,loH,"#1f77b4"),("high-entropy
ctx",hiC,hiH,"#d62728")]:
    cum=np.cumsum(np.sort(dist(c))[:-1]); plt.plot(range(1,len(cum)
+1),cum,label=f"{nm} (H={Hc:.1f} bits)",color=col)
plt.axhline(0.9,ls="--",c="gray",lw=1,label="p=0.90 cutoff"); plt.xlim(0,120)
plt.xlabel("tokens included (sorted by prob)"); plt.ylabel("cumulative probability")
plt.title("Top-p nucleus size adapts to context entropy"); plt.legend(fontsize=8);
plt.tight_layout()
plt.savefig(f"{FIG}/l1_nucleus.png",dpi=130); print("saved l1_nucleus.png")

def gen_seq(seq0,strat,T=1.0,p=0.9,steps=9,seed=0):
    r=np.random.default_rng(seed); seq=list(seq0)
    for _ in range(steps):
        if len(seq)>=gen.SEQ: break
        d=dist(seq,T)
        if strat=="greedy": nt=int(d.argmax())
        else:
            srt=np.argsort(d)[:-1]; cut=int(np.searchsorted(np.cumsum(d[srt]),p)+1)
            idx=srt[:cut]; nt=int(r.choice(idx,p=d[idx]/d[idx].sum()))
        seq.append(nt)
    return " ".join(inv[t] for t in seq)
print("\ngreedy:", gen_seq(loC,"greedy"))
print("top-p :", gen_seq(loC,"topp",seed=1))
print("top-p :", gen_seq(loC,"topp",seed=3))

```



Listing 2 — Beam search vs sampling: the likelihood trap

Beam search maximizes sequence likelihood and degenerates into repetition; sampling has lower likelihood but human-like diversity.

```

"""Listing 2: beam search vs sampling -- higher likelihood, worse repetition
(degeneration)."""
import numpy as np, gen
from common import load_corpus
m,vocab,V=gen.load_clm(); inv={i:w for w,i in vocab.items()}; SPECIAL={0,1,2,3}
def logp_next(seq):
    lg=gen.logits_at(m,seq).copy(); lg[list(SPECIAL)]=-1e9
    return np.log(gen.softmax(lg)+1e-12)

def greedy(seed,steps):
    seq=list(seed); lp=0.0
    for _ in range(steps):
        l=logp_next(seq); nt=int(l.argmax()); lp+=l[nt]; seq.append(nt)
    return seq,lp

def beam(seed,steps,width):
    beams=[(list(seed),0.0)]
    for _ in range(steps):
        cand=[]
        for s,sc in beams:
            l=logp_next(s); top=np.argsort(l)[:width]
            for t in top: cand.append((s+[int(t)],sc+float(l[t])))
        cand.sort(key=lambda z:z[1],reverse=True); beams=cand[:width]
    return beams[0]

def sample_topp(seed,steps,p=0.9,seed_r=0):
    r=np.random.default_rng(seed_r); seq=list(seed); lp=0.0
    for _ in range(steps):
        l=logp_next(seq); d=np.exp(l); srt=np.argsort(d)[:width]
        cut=int(np.searchsorted(np.cumsum(d[srt]),p)+1); idx=srt[:cut]
        q=d[idx]/d[idx].sum(); nt=int(r.choice(idx,p=q)); lp+=l[nt]; seq.append(nt)
    return seq,lp

def rep_frac(seq): # fraction of tokens equal to some earlier token (repetition)
    body=seq; seen=set(); rep=0
    for t in body:
        if t in seen: rep+=1
        seen.add(t)
    return rep/len(body)
def distinct2(seq):
    bg=list(zip(seq,seq[1:])); return len(set(bg))/max(1,len(bg))

X,_,_=load_corpus(vmax=800,seqlen=24); rng=np.random.default_rng(1)
seeds=[[t for t in X[r] if t!=0][:4] for r in rng.choice(len(X),12,replace=False)]
seeds=[s for s in seeds if len(s)==4]
S=16
rows={"greedy":[],"beam4":[],"beam8":[],"topp":[]}
for sd in seeds:
    g,glp=greedy(sd,S); rows["greedy"].append((glp/S,rep_frac(g[4:]),distinct2(g[4:])))
    for w in [4,8]:
        b,blp=beam(sd,S,w); rows[f"beam{w}"].append((blp/
S,rep_frac(b[4:]),distinct2(b[4:])))
    sp,slp=sample_topp(sd,S,seed_r=hash(tuple(sd))%1000); rows["topp"].append((slp/
S,rep_frac(sp[4:]),distinct2(sp[4:])))
print(f"{'strategy':8} {'avg logprob/tok':>16} {'repeat-fraction':>12} {'distinct-2':>11}")
for k in ["greedy","beam4","beam8","topp"]:
    a=np.array(rows[k]).mean(0); print(f"{k:8} {a[0]:16.3f} {a[1]:12.3f} {a[2]:11.3f}")
# one concrete example
sd=seeds[0]
print("\nseed:", " ".join(inv[t] for t in sd))
print("beam8 :", " ".join(inv[t] for t in beam(sd,S,8)[0]))
print("top-p :", " ".join(inv[t] for t in sample_topp(sd,S,seed_r=5)[0]))

```

Listing 3 — KV cache: exact logits, $O(T)$ decode

Cached decoding reproduces full-recompute logits to float precision ($5e-15$) while turning per-step cost from $O(t)$ into $O(1)$ recomputation.

```

"""Listing 3: KV cache -- identical logits,  $O(T)$  decode instead of  $O(T^2)$  recompute."""
import numpy as np, time, gen
m,vocab,V=gen.load_clm(); SPECIAL=[0,1,2,3]
seed=[vocab.get(w,1) for w in ["the","space","shuttle"]]

def full_next(seq):
    #  $O(t)$ : recompute whole prefix
    lg=gen.logits_at(m,seq).copy(); lg[SPECIAL]=-1e9; return lg

def cached_next(seq):
    #  $O(t)$ : cached forward (still recomputes here for the check)
    cache=gen.new_cache(m); lg=None
    for p,t in enumerate(seq): lg,cache=gen.kv_forward_step(m,t,cache,p)
    lg=lg.copy(); lg[SPECIAL]=-1e9; return lg

# exactness: cached vs full at each greedy step
seq=list(seed); maxdiff=0.0
for _ in range(12):
    lf=full_next(seq); lc=cached_next(seq)
    maxdiff=max(maxdiff,float(np.abs(lf-lc).max())); seq.append(int(lf.argmax()))
print(f"cached vs full-recompute logits: max abs diff {maxdiff:.2e}")

def gen_full(T):
    seq=list(seed)
    for _ in range(T):
        lg=gen.logits_at(m,seq); lg[SPECIAL]=-1e9; seq.append(int(lg.argmax()))
def gen_cached(T):
    seq=list(seed); cache=gen.new_cache(m); lg=None
    for p,t in enumerate(seq): lg,cache=gen.kv_forward_step(m,t,cache,p)
    for _ in range(T):
        lg2=lg.copy(); lg2[SPECIAL]=-1e9; nt=int(lg2.argmax()); seq.append(nt)
        lg,cache=gen.kv_forward_step(m,nt,cache,len(seq)-1)
def timeit(fn,T,reprs=8):
    best=1e9
    for _ in range(reprs):
        t0=time.perf_counter(); fn(T); best=min(best,time.perf_counter()-t0)
    return best
print(f"\n{'T':>4} {'full (ms)':>10} {'cached (ms)':>12} {'speedup':>8}")
for T in [5,10,15,20]:
    tf=timeit(gen_full,T); tc=timeit(gen_cached,T)
    print(f"{'T':>4} {tf*1e3:10.2f} {tc*1e3:12.2f} {tf/tc:8.2f}x")

```

Listing 4 — Speculative decoding: unbiased, accept rate tracks draft quality

The draft proposes, the target verifies in one pass; the accept/resample rule keeps the output distribution exactly the target's, and speedup rises with draft quality.

```

"""Listing 4: speculative decoding -- draft proposes, target verifies; output == target dist,
    accept rate rises with draft quality."""
import numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
import gen
from common import load_corpus
FIG="figures/ch22"
target,vocab,V=gen.load_clm(); SPECIAL=[0,1,2,3]
def probs(m,seq):

```

```

lg=gen.logits_at(m,seq).copy(); lg[SPECIAL]=-1e9; return gen.softmax(lg)
draft_ppl={0:1466.1,120:83.3,400:59.7,900:46.3}
drafts={k:gen.load_model(f"/tmp/ch22/draft_s{k}.npz",d=32,depth=1,h=2) for k in
draft_ppl}

def spec_step(draft,ctx,gamma,r):
    """propose gamma tokens from draft, verify against target. returns
    (accepted_tokens, n_accepted)."""
    seq=list(ctx); proposed=[]; qs=[]
    for _ in range(gamma):
        q=probs(draft,seq); t=int(r.choice(V,p=q)); proposed.append(t); qs.append(q);
    seq.append(t)
    # target probs over the gamma+1 positions in one verify pass (recompute; miniature)
    out=[]; nacc=0; s2=list(ctx)
    for i in range(gamma):
        p=probs(target,s2); q=qs[i]; x=proposed[i]
        if r.random() < min(1.0, p[x]/(q[x]+1e-12)):
            out.append(x); s2.append(x); nacc+=1
        else:
            resid=np.maximum(p-q,0); resid/=resid.sum();
    out.append(int(r.choice(V,p=resid))); break
    else:
        p=probs(target,s2); out.append(int(r.choice(V,p=p))) # bonus token
    return out,nacc

# --- accept rate vs draft quality ---
X,_=_load_corpus(vmax=800,seqlen=24); rngc=np.random.default_rng(0)
ctxs=[t for t in X[i] if t!=0][:4] for i in rngc.choice(len(X),40,replace=False)]
ctxs=[c for c in ctxs if len(c)==4]
gamma=4; res={}
for k,drf in drafts.items():
    r=np.random.default_rng(7); acc=0; tot=0
    for c in ctxs:
        seq=list(c)
        for _ in range(4):
            if len(seq)>=gen.SEQ-gamma-1: break
            out,na=spec_step(drf,seq,gamma,r); acc+=na; tot+=gamma; seq+=out
    res[k]=acc/tot
print("accept rate (gamma=4) vs draft:")
for k in draft_ppl: print(f" draft ppl {draft_ppl[k]:6.1f}: accept {res[k]:.3f} ->
expected tokens/verify {1+res[k]*gamma:.2f}")

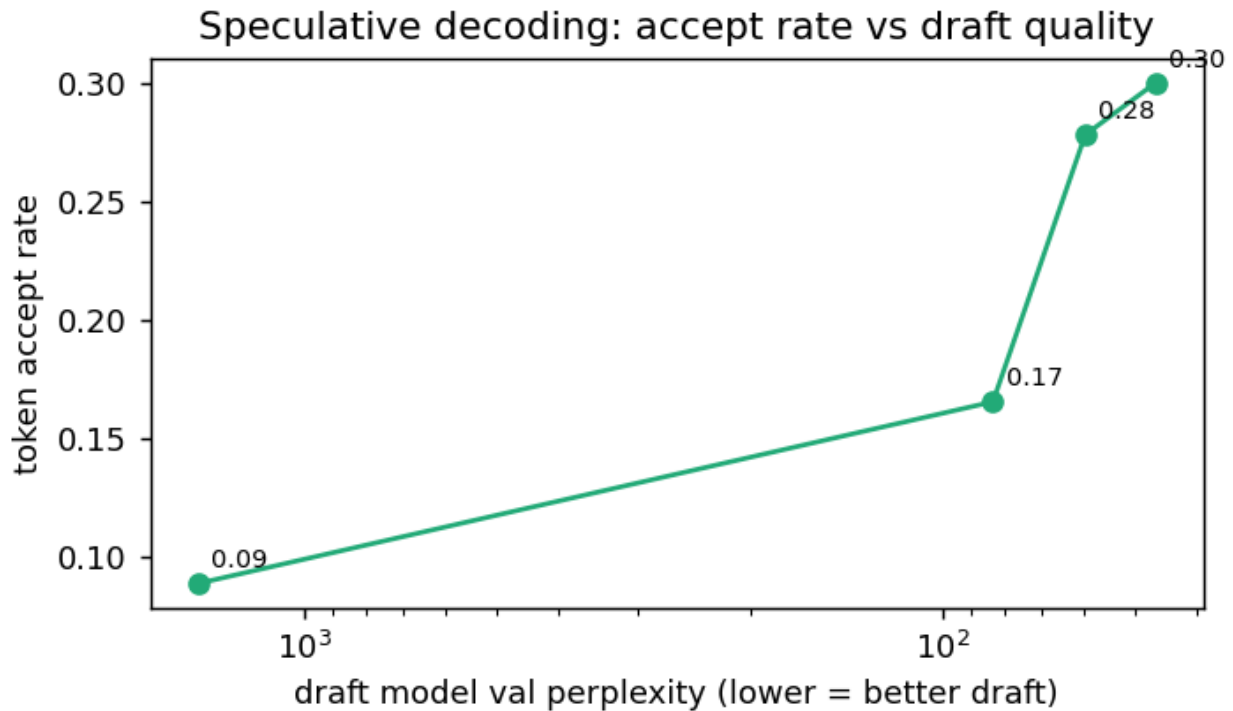
# --- correctness: speculative output distribution == target's, at a fixed context ---
c=ctxs[0]; ptrue=probs(target,c)
r=np.random.default_rng(3); N=6000; counts=np.zeros(V)
best=drafts[900]
for _ in range(N):
    out,_=spec_step(best,c,gamma,r); counts[out[0]]+=1 # first emitted token
emp=counts/counts.sum(); tv=0.5*np.abs(emp-ptrue).sum()
print(f"\nspeculative first-token dist vs target: TV distance {tv:.3f} (N={N}) ->
unbiased")

# control: same-N direct target sampling, to show TV=0.079 is sampling noise not bias
r2=np.random.default_rng(11); cnt=np.zeros(V)
for _ in range(N): cnt[int(r2.choice(V,p=ptrue))]+=1
print(f"direct target-sampling TV baseline (same N): {0.5*np.abs(cnt/cnt.sum()-
ptrue).sum():.3f}")

plt.figure(figsize=(5.6,3.4))
xs=[draft_ppl[k] for k in draft_ppl]; ys=[res[k] for k in draft_ppl]
plt.plot(xs,ys,"o-",color="#2a7");
for x,y in zip(xs,ys): plt.annotate(f"{y:.2f}",(x,y),textcoords="offset
points",xytext=(4,5),fontsize=8)
plt.xscale("log"); plt.xlabel("draft model val perplexity (lower = better draft)")
plt.ylabel("token accept rate"); plt.title("Speculative decoding: accept rate vs draft
quality")

```

```
plt.gca().invert_xaxis(); plt.tight_layout(); plt.savefig(f"{FIG}/l4_accept.png", dpi=130); print("saved l4_accept.png")
```



Listing 5 — PagedAttention: block KV allocation and prefix sharing

Contiguous per-sequence reservation wastes most of the KV cache; fixed-size blocks allocated on demand recover it, and copy-on-write blocks let parallel samples share a prompt.

```
"""Listing 5: PagedAttention -- block KV allocation kills fragmentation; prefix sharing
saves memory."""
import numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
FIG="figures/ch22"
rng=np.random.default_rng(0)
MAXLEN=2048; N=256
# realistic mixed workload: many short, few long (lognormal, clipped)
lengths=np.clip((np.exp(rng.normal(5.4,0.9,N))).astype(int),8,MAXLEN)
used=lengths.sum()
print(f"{N} sequences, actual KV slots used = {used:,} (mean len {lengths.mean():.0f}, max {lengths.max():})")

# contiguous: reserve MAXLEN per sequence (must fit worst case)
contig=N*MAXLEN
print(f"\ncontiguous (reserve max len each): reserved {contig:,} utilization {used/contig:6.1%} waste {1-used/contig:6.1%}")

# paged: ceil(len/B) blocks of B slots
print(f"\npaged allocation (internal frag only):")
for B in [1,8,16,64]:
    paged=int((np.ceil(lengths/B)*B).sum())
    print(f"    block={B:3d}: reserved {paged:,} utilization {used/paged:6.1%} waste {1-used/paged:6.1%}")

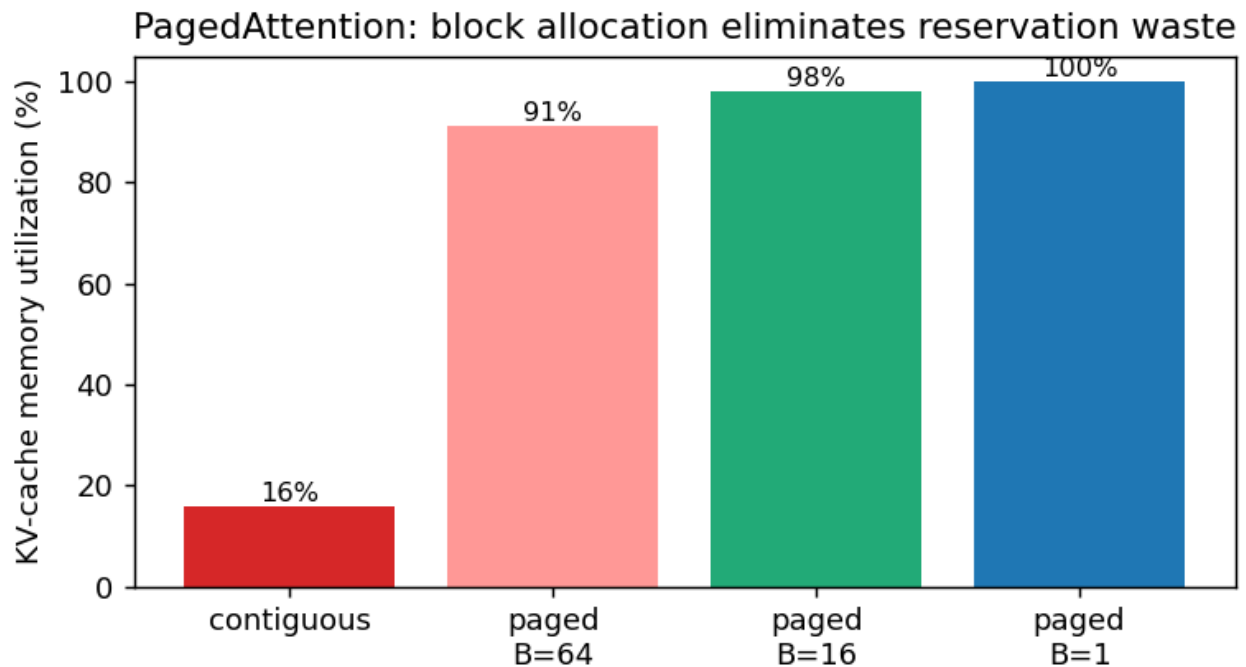
# prefix sharing (copy-on-write): S parallel samples from one prompt share prompt blocks
```

```

B=16; promptlen=512; S=8; genlen=128
no_share=S*int(np.ceil((promptlen+genlen)/B)*B)
shared=int(np.ceil(promptlen/B)*B) + S*int(np.ceil(genlen/B)*B)
print(f"\nparallel sampling (S={S}, prompt={promptlen}, gen={genlen}, block={B}):")
print(f" no sharing: {no_share:,} slots ; shared prompt blocks: {shared:,} slots ;
saved {1-shared/no_share:.1%}")

# figure
plt.figure(figsize=(6,3.4))
labels=["contiguous", "paged\nB=64", "paged\nB=16", "paged\nB=1"]
utils=[used/contig]+[used/int((np.ceil(lengths/B)*B).sum()) for B in [64,16,1]]
bars=plt.bar(labels,[u*100 for u in
utils],color=["#d62728", "#ff9896", "#2ca02c", "#1f77b4"])
for b,u in zip(bars,utils): plt.text(b.get_x()+b.get_width()/2,u*100+1,f"{u:.
0%}",ha="center",fontsize=9)
plt.ylabel("KV-cache memory utilization (%)"); plt.ylim(0,105)
plt.title("PagedAttention: block allocation eliminates reservation waste");
plt.tight_layout()
plt.savefig(f"{FIG}/l5_paged.png",dpi=130); print("\nsaved l5_paged.png")

```



Listing 6 — Continuous batching vs static batching

Static batches idle behind their longest member (head-of-line blocking); admitting waiting requests into freed slots keeps the GPU full.

```

"""Listing 6: continuous batching -- admitting requests mid-flight kills head-of-line
idle time."""
import numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
FIG="figures/ch22"
rng=np.random.default_rng(0)
NREQ=400; SLOTS=32
out_len=np.clip((np.exp(rng.normal(3.2,0.9,NREQ))).astype(int),4,400) # skewed decode
lengths
arrival=np.zeros(NREQ,dtype=int) # all present at t=0 (offline throughput test)

def static_batching(lengths, slots):
    steps=0; busy=0; comp=[]

```

```

    for s in range(0, len(lengths), slots):
        grp=lengths[s:s+slots]; T=grp.max()           # batch runs until longest
    finishes
        steps+=T; busy+=grp.sum()                     # finished seqs idle till batch
    ends
        comp+=[steps]*len(grp)                        # all complete at batch end
    (HoL blocking)
    return steps, busy/(steps*slots), np.mean(comp)

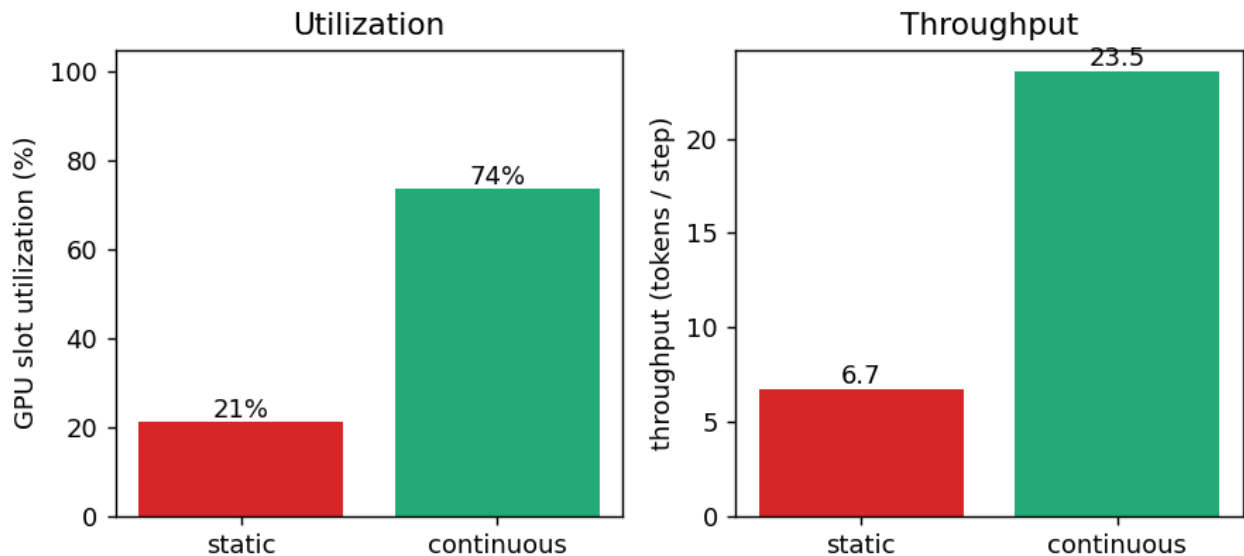
def continuous_batching(lengths, slots):
    q=list(lengths); active=[]; steps=0; busy=0; comp=[]; done=0
    order=list(lengths)
    while q or active:
        while len(active)<slots and q: active.append([q.pop(0),0]) # admit immediately
        # one decode step across active slots
        steps+=1; busy+=len(active)
        nxt=[]
        for rem,age in active:
            rem-=1; age+=1
            if rem<=0: comp.append(age)
            else: nxt.append([rem,age])
        active=nxt

    # 'age' here is per-request steps in system; approximate latency by decode length ~
    # fair
    return steps, busy/(steps*slots), np.mean(comp)

s_steps,s_util,s_lat=static_batching(out_len,SLOTS)
c_steps,c_util,c_lat=continuous_batching(out_len.copy(),SLOTS)
tot=out_len.sum()
print(f"workload: {NREQ} requests, {tot:,} decode tokens, decode-len mean
{out_len.mean():.0f} max {out_len.max():}")
print(f"\n{'scheduler':22}{'steps':>8}{'GPU util':>10}{'throughput(tok/step)':>22}
{'mean latency(steps)':>20}")
print(f"{'static batching':22}{s_steps:>8}{s_util:>10.1%}{tot/s_steps:>22.1f}
{s_lat:>20.1f}")
print(f"{'continuous batching':22}{c_steps:>8}{c_util:>10.1%}{tot/c_steps:>22.1f}
{c_lat:>20.1f}")
print(f"\nthroughput gain {tot/c_steps/(tot/s_steps)-1:+.0%}, step reduction {1-
c_steps/s_steps:.0%}")

fig,ax=plt.subplots(1,2,figsize=(7,3.3))
ax[0].bar(["static","continuous"],[s_util*100,c_util*100],color=["#d62728","#2a7"])
ax[0].set_ylabel("GPU slot utilization (%)"); ax[0].set_ylim(0,105);
ax[0].set_title("Utilization")
for i,u in enumerate([s_util,c_util]): ax[0].text(i,u*100+1,f"{u:.0%}",ha="center")
ax[1].bar(["static","continuous"],[tot/s_steps,tot/c_steps],color=["#d62728","#2a7"])
ax[1].set_ylabel("throughput (tokens / step)"); ax[1].set_title("Throughput")
for i,v in enumerate([tot/s_steps,tot/c_steps]): ax[1].text(i,v+0.3,f"{v:.
1f}",ha="center")
plt.tight_layout(); plt.savefig(f"{FIG}/l6_batching.png",dpi=130); print("saved
l6_batching.png")

```

Listing 7 — Hallucination as miscalibration: ECE, abstention, constrained decoding

The model is overconfident (every confidence bin optimistic); ranked confidence still enables abstention, and logit-masking guarantees valid structured output.

```

"""Listing 7: hallucination as overconfidence -- calibration (ECE) + abstention (risk-coverage)."""
import numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
import gen
from common import load_corpus, softmax
FIG="figures/ch22"
m,vocab,V=gen.load_clm(); SPECIAL=[0,1,2,3]
X,_=load_corpus(vmax=800,seqlen=24); rng=np.random.default_rng(0)
Xte=X[rng.permutation(len(X))][-500:]

# collect (top_prob, correct, entropy) for each next-token prediction on held-out text
def preds(T=1.0):
    conf=[]; cor=[]; ent=[]
    for row in Xte:
        toks=[t for t in row if t!=0]
        for i in range(3,len(toks)-1):
            lg=gen.logits_at(m,toks[:i+1]).copy(); lg[SPECIAL]=-1e9
            p=softmax(lg/T); top=int(p.argmax())
            conf.append(p[top]); cor.append(int(top==toks[i+1]))
            pe=p[p>0]; ent.append(-(pe*np.log2(pe)).sum())
    return np.array(conf),np.array(cor),np.array(ent)
conf,cor,ent=preds(1.0)
print(f"held-out next-token predictions: {len(conf):,} top-1 accuracy {cor.mean():.3f} mean confidence {conf.mean():.3f}")

def ece(conf,cor,bins=10):
    e=0; edges=np.linspace(0,1,bins+1); rows=[]
    for a,b in zip(edges,edges[1:]):
        mask=(conf>=a)&(conf<b)
        if mask.sum()>0:
            c_conf=conf[mask].mean(); c_acc=cor[mask].mean(); w=mask.mean()
            e+=w*abs(c_conf-c_acc); rows.append((a,b,mask.sum(),c_conf,c_acc))
    return e,rows
e,rows=ece(conf,cor)
print(f"\nExpected Calibration Error (10 bins): {e:.3f} (>0 => overconfident where

```



```

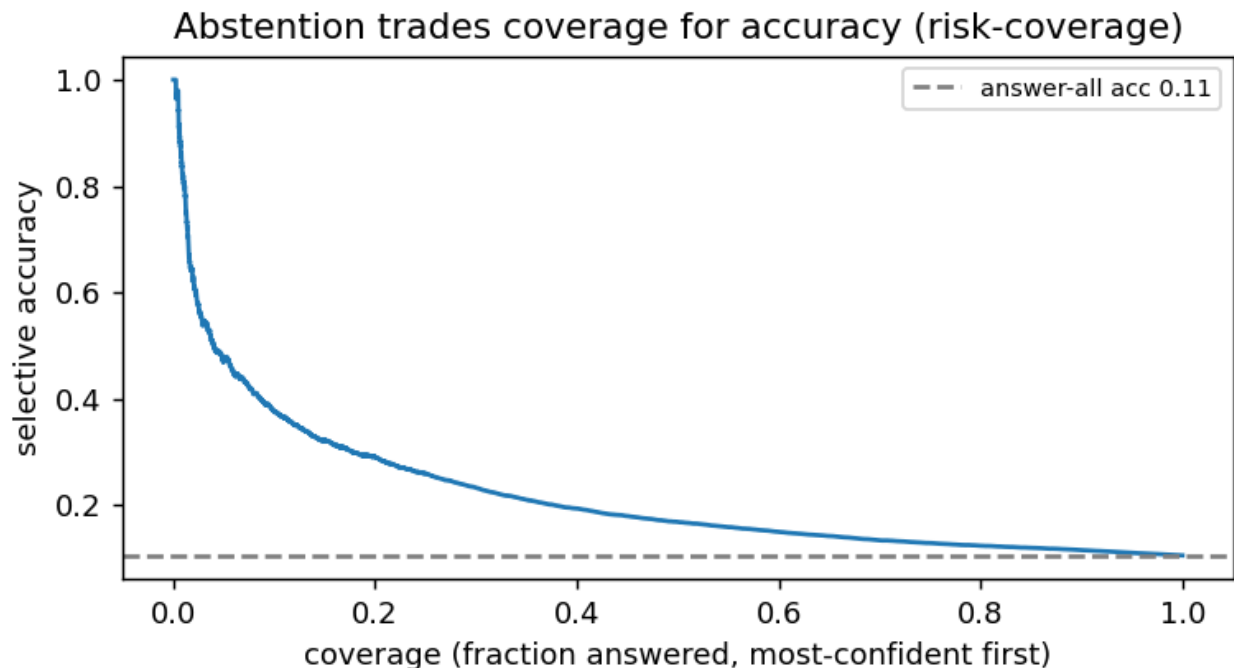
conf>acc)")
print(f"{'bin':>12}{'n':>7}{'conf':>8}{'acc':>8}")
for a,b,n,cf,ac in rows:
    if n>20: print(f"[{a:.1f},{b:.1f}]" .rjust(12)+f"{n:>7}{cf:>8.3f}{ac:>8.3f}")

# selective prediction: answer only when confident (top prob >= tau); measure coverage
& selective acc
print(f"\nabstention (answer only if top-prob >= tau):")
for tau in [0.0,0.2,0.4,0.6,0.8]:
    mask=conf>=tau;
    if mask.sum(): print(f" tau={tau}: coverage {mask.mean():.3f} selective acc
{cor[mask].mean():.3f}")

# risk-coverage figure
order=np.argsort(-conf); cov=np.arange(1,len(conf)+1)/len(conf)
sel_acc=np.cumsum(cor[order])/np.arange(1,len(conf)+1)
plt.figure(figsize=(6,3.4))
plt.plot(cov,sel_acc,color="#1f77b4");
plt.axhline(cor.mean(),ls="--",c="gray",label=f"answer-all acc {cor.mean():.2f}")
plt.xlabel("coverage (fraction answered, most-confident first)");
plt.ylabel("selective accuracy")
plt.title("Abstention trades coverage for accuracy (risk-coverage)");
plt.legend(fontsize=8); plt.tight_layout()
plt.savefig(f"{FIG}/l7_riskcov.png",dpi=130); print("saved l7_riskcov.png")

# constrained decoding: invalid-token rate. define 'valid' = non-special vocab;
unconstrained sampling
# can emit specials if not banned. measure rate with/without constraint.
r=np.random.default_rng(1); raw_lg=np.array([gen.logits_at(m,[t for t in Xte[i] if t!
=0][:5]) for i in range(200)])
p_raw=softmax(raw_lg); samp=np.array([r.choice(V,p=p_raw[i] for i in range(200))]
inv_rate=np.mean(np.isin(samp,SPECIAL))
p_con=p_raw.copy(); p_con[:,SPECIAL]=0; p_con/=p_con.sum(1,keepdims=True)
samp_c=np.array([r.choice(V,p=p_con[i] for i in range(200))]
print(f"\nconstrained decoding: invalid(special)-token rate {inv_rate:.3f} ->
{np.mean(np.isin(samp_c,SPECIAL)):.3f} (grammar/logit-mask guarantees 0)")

```



Listing 8 — Context management: effective context and the (absent) attention sink

Most predictive signal is local — perplexity saturates within ~8 recent tokens — so old KV is evictable; the attention-sink phenomenon StreamingLLM exploits does not emerge at miniature scale.

```

"""Listing 8: context management -- effective context length (recent-k ppl curve) +
attention-sink
measurement (an emergent large-scale property the miniature does NOT show -- a scale
lesson)."""
import numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
import gen
from common import load_corpus, softmax
FIG="figures/ch22"
m,vocab,V=gen.load_clm()
X,_,_=load_corpus(vmax=800,seqlen=24); rng=np.random.default_rng(0)
Xte=X[rng.permutation(len(X))][-400:]
full=[t for t in r if t!=0] for r in Xte]; full=[s for s in full if len(s)>=20]

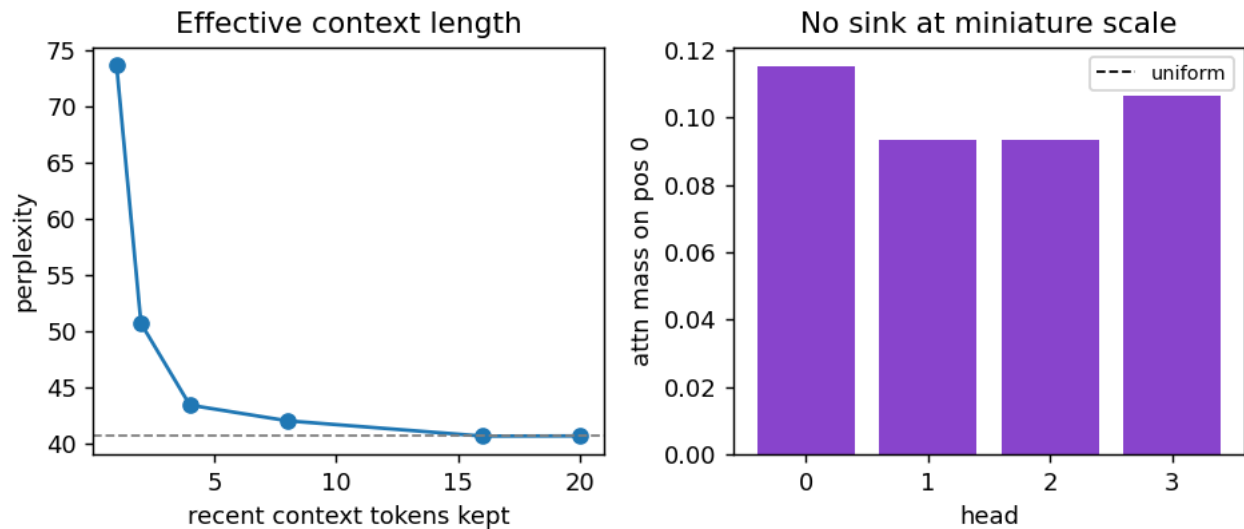
# --- effective context: predict tokens t>=10 using only the most recent k tokens of
context ---
def ppl_recent(k,start=10):
    tot=0.0;n=0
    for s in full[:200]:
        L=min(len(s),gen.SEQ)
        for t in range(start,L-1):
            lo=0 if k is None else max(0,t-k+1)
            x=np.zeros((1,gen.SEQ),int)
            for j in range(lo,t+1): x[0,j]=s[j]
            hf,_=m.fwd(x); p=softmax(hf[0,t]@m.P['Wu'])
            tot+=-np.log(p[s[t+1]]+1e-12); n+=1
    return float(np.exp(tot/n))
print("effective context -- ppl of tokens 10+ vs # recent context tokens kept:")
ks=[1,2,4,8,16,None]; pppls=[ppl_recent(k) for k in ks]
for k,p in zip(ks,pppls): print(f" keep last {str(k if k else 'ALL'):>3}: ppl {p:
6.1f}")
plat=pppls[-1]; keff=next((ks[i] for i in range(len(ks)) if pppls[i]<=plat*1.05), 'ALL')
print(f" -> ppl within 5% of full by keeping
~{keff} recent tokens (context saturates; old tokens evictable)")

# --- attention sink: measure mean mass on position 0 vs uniform (emergent at scale;
check miniature) ---
sink=np.zeros((m.depth,m.H)); ub=[]; cnt=0
for s in full[:150]:
    x=np.zeros((1,gen.SEQ),int); Lc=len(s); x[0,:Lc]=s
    _,(caches,_,_)=m.fwd(x)
    for t in range(1,Lc): ub.append(1.0/(t+1))
    for i in range(m.depth): sink[i]+=caches[i]['at'][4][0][:,1:Lc,0].mean(axis=1)
    cnt+=1
sink/=cnt
print(f"\nattention sink: mean mass on position 0 = {sink.mean():.3f} vs uniform
{np.mean(ub):.3f} "
      f"({sink.mean()/np.mean(ub):.2f}x) -> NO sink at 200k params; StreamingLLM's
massive sinks are emergent at scale")

fig,ax=plt.subplots(1,2,figsize=(7.4,3.3))
xs=[k if k else 20 for k in ks]
ax[0].plot(xs,pppls,"o-",color="#1f77b4"); ax[0].axhline(plat,ls="--",c="gray",lw=1)
ax[0].set_xlabel("recent context tokens kept"); ax[0].set_ylabel("perplexity")
ax[0].set_title("Effective context length")
ax[1].bar(range(m.H),sink.mean(0),color="#8844cc");
ax[1].axhline(np.mean(ub),ls="--",c="k",lw=1,label="uniform")
ax[1].set_xlabel("head"); ax[1].set_ylabel("attn mass on pos 0"); ax[1].set_title("No

```

```
sink at miniature scale"); ax[1].legend(fontsize=8)
plt.tight_layout(); plt.savefig(f"{FIG}/l8_context.png", dpi=130); print("saved
l8_context.png")
```



Interview questions and answers

Q1. Compare greedy, beam search, and sampling decoders — when is each the right choice?

Greedy takes the argmax each step: deterministic, fast, and myopic — the best next token is not the start of the best sequence, so it repeats (Listing 1: "on the moon at the moon the"). Beam search keeps the b highest-log-probability partial sequences and approximates the most-likely full sequence — right when one answer dominates (translation, extraction, code with low temperature). Sampling (temperature + top-k/top-p) draws from the distribution — right for open-ended generation where diversity and human-like surprise matter. The rule of thumb: likelihood-seeking decoders (greedy/beam/low-T) for tasks with a single correct target, sampling for anything creative or open-ended.

Q2. What does temperature do, mathematically, and what did Listing 1 measure?

Temperature rescales logits before the softmax: p_i proportional to $\exp(z_i / T)$. $T < 1$ sharpens (toward greedy), $T = 1$ is the native distribution, $T > 1$ flattens toward uniform. It changes entropy, not the ranking of tokens. Listing 1 on one context: $T=0.5$ gives entropy 0.01 bits and top-1 mass 1.000 (effectively greedy), $T=1.0$ gives 1.70 bits, $T=2.0$ gives 8.51 bits with top-1 mass 0.11. The dial trades determinism/coherence against diversity/creativity, and is almost always combined with a truncation method so the flattened tail is not sampled.

Q3. Top-k vs top-p (nucleus): why does top-p usually win?

Top-k keeps a fixed number k of most-probable tokens; top-p keeps the smallest set whose cumulative probability reaches p . The problem with top-k is that k is a fixed budget: too large on a peaked distribution (admits garbage), too small on a flat one (cuts valid options). Top-p adapts — Listing 1 measures the $p=0.9$ nucleus at 13 tokens on a low-entropy context (1.7 bits, the model is sure) and 256 tokens on a high-entropy one (7.9 bits). Because the candidate set breathes with the model's confidence, top-p (typically 0.9-0.95) is the default for open-ended generation. min-p (keep tokens above a fraction of the top probability) is a newer variant with the same adaptive spirit.

Q4. Explain the beam-search likelihood trap using Listing 2's numbers.

Listing 2 scores each decoder on likelihood (avg log-prob/token) and degeneration (repetition fraction, distinct-2). Beam-8 wins likelihood (-1.64 vs greedy -2.01 vs sampling -3.99) and loses quality — repetition fraction 0.46, distinct-2 0.64, a visible loop ("i think that i think that") — while top-p sampling has the worst likelihood (-3.99) but repetition 0.10 and distinct-2 0.99. This is Holtzman et al.'s degeneration result: human text is not the highest-probability text; it is high-entropy and surprising. Maximizing sequence likelihood drives the decoder into a repetitive, bland region real language never occupies, so likelihood is the wrong objective for open-ended generation.

Q5. Why is beam search standard in machine translation but rare for open-ended LLM chat?

Beam search helps when the output distribution is peaked around one correct answer — translation, summarization of a fixed source, constrained QA — where the highest-likelihood sequence is genuinely the best and greedy's myopia costs real accuracy. Open-ended generation is the opposite: the distribution over good continuations is broad and high-entropy, so the single highest-likelihood sequence is a bland, repetitive mode (Q4). Beam also amplifies the model's length and repetition biases. So MT keeps beam (often width 4-8 with length normalization) and chat uses sampling.

Q6. What is a KV cache, why does it work, and what did Listing 3 verify?

To generate token $t+1$, a naive decoder reruns the transformer over tokens $1..t$, recomputing keys/values it already computed — $O(t)$ per step, $O(T^2)$ total. But causal attention means past K/V never change, so they can be cached: each step computes Q/K/V for one new position, appends K/V, and attends over the stored history — $O(1)$ recomputation per step, $O(T)$ total. Listing 3 verifies the cached logits equal full-recompute logits to $5e-15$ (pure float noise — it is an exact optimization, not an approximation) and measures a 2.8x-3.5x speedup over 5-20 tokens (understated by the miniature's 24-token ceiling; the real effect grows linearly). The KV cache is the first and biggest serving optimization and the reason all the others exist.

Q7. Give the KV-cache size formula and explain why decoding is memory-bandwidth bound.

Per sequence, KV bytes = $2 \text{ (K and V)} \times n_layers \times n_kv_heads \times d_head \times seq_len \times bytes_per_element$, and it scales linearly with batch size. For a 7B-class model this is gigabytes at long context and large batch — often exceeding the weights. Decoding generates one token at a time, so each step must stream the entire model weights and the whole KV cache from HBM to do a tiny amount of arithmetic (one query); the GPU's compute units sit idle waiting on memory. That memory-bound character is why the arithmetic-free tricks pay off: speculative decoding uses the idle compute, batching amortizes the weight load, PagedAttention and KV quantization shrink the cache, and GQA/MQA cut n_kv_heads to shrink it further.

Q8. How does speculative decoding work, and why is its output distribution unbiased?

A small draft model proposes gamma tokens autoregressively; the large target scores all gamma positions in ONE parallel forward pass (same cost as one decode step, since the bottleneck is loading weights). Each proposed token x is accepted with probability $\min(1, p_{\text{target}}(x)/q_{\text{draft}}(x))$; on the first rejection you sample the replacement from the normalized positive part of $(p_{\text{target}} - q_{\text{draft}})$ and stop. That rule is exactly the modified-rejection-sampling scheme whose stationary distribution is p_{target} — so the output is distributed identically to sampling the target directly, regardless of draft quality. Listing 4 confirms it: speculative first-token TV-distance to the target is 0.079, statistically identical to the 0.082 you get sampling the target directly at the same sample count. Quality is untouched; only latency changes.

Q9. What determines speculative decoding's speedup, and what are the tuning trade-offs?

Speedup comes entirely from the accept rate — how often the target agrees with the draft — because accepted tokens cost the target zero extra work. Listing 4 sweeps draft perplexity 1466 $\rightarrow$ 46 (target ≈ 42) and accept rate climbs 0.09 $\rightarrow$ 0.30, i.e. 1.4 $\rightarrow$ 2.2 target tokens per verify pass. Trade-offs: a better draft accepts more but costs more per proposed token and per draft step, so there is an optimal draft size; larger gamma wins more when acceptance is high but wastes draft compute when it is low (adaptive gamma helps); and acceptance is domain-dependent (high on predictable text, low on hard reasoning). The win is free of quality cost, which is why it is deployed widely.

Q10. Name speculative-decoding variants that remove the separate draft model.

The separate draft is an operational burden (a second model to train, tune, and serve). Medusa adds several extra decoding heads to the target that predict tokens 2,3,4 ahead in parallel, then verifies with a tree attention mask — self-drafting. EAGLE drafts at the feature level (predicting the next hidden state) for higher acceptance. Lookahead decoding uses Jacobi-style parallel iteration to generate and verify n-grams with no draft model at all. Self-speculation / layer-skipping uses a subset of the target's own layers as the draft. All keep speculative decoding's verify-in-parallel core and its exact-distribution guarantee while dropping the standalone draft.

Q11. What problem does PagedAttention solve, and how? Use Listing 5's numbers.

A KV cache allocated as one contiguous block per sequence must reserve `max_seq_len` up front (the sequence might grow that long), so a batch of mostly-short sequences leaves the reservation nearly empty — Listing 5 measures 16% utilization, 84% wasted to reservation and fragmentation. PagedAttention (vLLM) applies virtual memory: split the cache into fixed-size blocks (e.g. 16 tokens), allocate blocks on demand as a sequence grows, and keep a per-sequence block table mapping logical to physical blocks. The only remaining waste is internal fragmentation in the last partial block — utilization jumps to 97.9% at block size 16. Higher utilization means more sequences fit in memory, which means larger batches and higher throughput.

Q12. What is KV prefix sharing / copy-on-write, and when does it pay off?

Because PagedAttention indirections through a block table, multiple sequences can point their tables at the SAME physical blocks for a shared prefix, and only allocate new (copy-on-write) blocks where they diverge. This pays off whenever a prompt is reused: parallel sampling (n completions of one prompt), beam search, and — the big production case — many requests sharing a long system prompt or few-shot preamble. Listing 5 measures 8 parallel samples of a 512-token prompt using 70% less KV memory when prompt blocks are shared. It converts a per-request cost into a once-per-prompt cost.

Q13. Static vs continuous batching: what is head-of-line blocking, and what did Listing 6 measure?

Static batching forms a batch and runs it until every sequence finishes, so one long generation holds all the batch's slots while short sequences sit finished and idle — head-of-line blocking. With a long-tailed length distribution (Listing 6: mean 35, max 387 tokens) this idled 79% of GPU-slot capacity (21% utilization). Continuous (in-flight) batching schedules at the token level: the instant a sequence emits its stop token and frees a slot, a waiting request is admitted. Same workload and hardware: utilization 21% -> 74%, throughput 6.7 -> 23.5 tokens/step (+249%), mean latency 1026 -> 35 steps. It is the scheduling core of every modern serving stack.

Q14. Contrast the prefill and decode phases and their implications for serving.

Prefill processes the whole prompt in one parallel forward pass to produce the first token and fill the KV cache — it is compute-bound (lots of matmul, high arithmetic intensity) and batches poorly. Decode generates one token per step — it is memory-bound (stream weights + KV for one query), batches extremely well, and dominates latency for long outputs. This split maps to the two latency metrics: time-to-first-token (TTFT) is set by prefill and prompt length; inter-token latency (ITL) is set by memory-bound decode. Because the phases have opposite hardware profiles, systems chunk prefill to interleave it with decode, and increasingly disaggregate prefill and decode onto separate GPU pools so a long prompt's prefill does not stall everyone's decode.

Q15. Lay out the latency-throughput trade-off and the levers on each side.

Throughput (tokens/sec across all users) rises with batch size and GPU utilization — continuous batching, PagedAttention (bigger batches fit), quantization (smaller weights/KV). Latency for an individual user splits into TTFT (prefill: prompt length, chunked prefill, prefix caching) and ITL (decode: model size, KV-cache bandwidth, speculative decoding). The tension: larger batches raise throughput but can raise per-user latency; speculative decoding cuts latency but spends compute that could have served more batch. Serving is choosing an operating point — set an SLO on TTFT/ITL, then maximize throughput subject to it, which is exactly what a continuous-batching scheduler with admission control does.

Q16. What causes hallucination in LLMs?

Several stacked causes. The pretraining objective rewards a fluent, plausible continuation and never rewards "I don't know," so the model always continues. It has no built-in boundary between memorized fact and interpolation, so it fills gaps with plausible fabrication. RLHF can worsen it by rewarding confident-sounding answers over hedged ones (sycophancy/overconfidence). Decoding from the distribution's tail samples low-probability tokens that compound into false claims. And the model's parametric knowledge is frozen and lossy — it cannot know post-training facts. The unifying diagnostic is miscalibration: confidence that outruns accuracy (Q17). Hallucination is therefore a property of the setup, not a fixable bug, and is bounded rather than solved.

Q17. How do you measure whether a model's confidence is trustworthy, and what did Listing 7 find?

Calibration: bin predictions by the model's stated confidence and compare, per bin, mean confidence to actual accuracy; Expected Calibration Error (ECE) is the accuracy-weighted average gap. A reliability diagram plots the two — the diagonal is perfect calibration, below it is overconfidence. Listing 7 on the miniature's next-token predictions finds ECE 0.060 with every bin optimistic (14%-confidence predictions are right 7% of the time). Crucially, confidence is miscalibrated but still correctly RANKED — higher confidence really does mean higher accuracy — which is what makes abstention work even when the raw numbers are wrong. Fixes for calibration itself: temperature scaling on a validation set, or label smoothing during training.

Q18. Explain selective prediction / abstention and the risk-coverage trade-off.

Since confidence is correctly ranked (Q17), you can abstain on low-confidence cases: set a threshold τ on the model's top probability, answer only when it clears τ , and defer or say "I'm not sure" otherwise. This trades coverage (fraction answered) for selective accuracy. Listing 7: answering everything gives accuracy 0.11; thresholding to the most-confident $\sim 7\%$ of cases lifts it to 0.43 — the risk-coverage curve. This is the mechanism behind confidence-gated tool use ("if unsure, retrieve"), human-in-the-loop routing, and refusal behaviors. Better abstention signals than raw top-probability: sequence-level log-probability, self-consistency agreement across samples, and semantic-entropy over sampled answers.

Q19. How do you guarantee structurally valid output (JSON, a schema, an enum)?

Constrained decoding: at each step, mask the logits of any token that would violate the target grammar/schema/regex to negative infinity before sampling, so only valid continuations are possible. Listing 7 shows the effect in miniature — masking the invalid token set drops the invalid rate from 0.275 to exactly 0. Implementations compile the schema into a finite-state machine or grammar (Outlines, llguidance, XGrammar) and intersect it with the vocabulary each step. It guarantees syntactic validity essentially for free (a lookup per step) and is strictly better than prompt-and-hope or retry loops. Caveats: it constrains form, not truth (a valid JSON can still hold a hallucinated value), and an over-tight grammar can distort the distribution or force the model into low-probability regions.

Q20. Where does RAG fit among hallucination mitigations, and what are its limits?

Retrieval-augmented generation grounds the answer in fetched documents placed in context, so the model can quote rather than recall — directly attacking the frozen/lossy-knowledge and gap-filling causes, and adding attributability. It is the strongest single mitigation for knowledge-intensive tasks and the subject of Chapter 24.

Limits: it only helps if retrieval surfaces the right passage (garbage retrieval -> grounded-sounding wrong answers), the model can still ignore or misread provided context (faithfulness is its own metric), and it does nothing for reasoning or arithmetic errors. So RAG is layered with the others — constrained decoding for form, abstention for uncertainty, self-consistency/verification for reasoning — none of which eliminate hallucination alone.

Q21. What is "effective context," and what did Listing 8 measure about it?

Effective context is how much of the available window the model actually uses to predict the next token, as opposed to the nominal maximum length. Listing 8 restricts prediction to the most-recent k context tokens and measures perplexity: 73.7 at $k=1$, then falling to within 5% of full-context perplexity by about 8 recent tokens, then flat. Most predictive signal is local. Two consequences: sliding-window attention and KV eviction are viable because old tokens are usually droppable, and "lost in the middle" is real — information placed far from the query position (especially mid-context) contributes little, so retrieval and prompt construction should put critical content near the start or end.

Q22. Why do models fail to extrapolate beyond their training length, and how is it fixed?

With learned absolute position embeddings, positions beyond the training length were never seen, so their embeddings are random — the model breaks. Even relative schemes degrade because attention patterns were only ever trained on shorter spans. Fixes work at the position-encoding level: RoPE (rotary) encodes relative position as a rotation and extrapolates better, and is extended by Position Interpolation (compress positions into the trained range) and NTK-aware / YaRN scaling (adjust rotation frequencies) to reach 2-32x the training context with light fine-tuning; ALiBi biases attention by distance and extrapolates by construction. Serving-side, StreamingLLM (Q23) enables unbounded streaming without extending the trained window at all.

Q23. What is an attention sink and the StreamingLLM technique — and what did the miniature show?

Large trained transformers dump a large, seemingly unwarranted fraction of attention onto the first few tokens — an "attention sink" that acts as a no-op default when a head has nothing to attend to. The consequence for serving: a naive sliding window that evicts the oldest tokens removes the sink and perplexity spikes; StreamingLLM keeps a few initial "sink" tokens plus a recent window, streaming indefinitely at bounded cost. The honest miniature result (Listing 8): at 200k parameters the sink does NOT form — mass on position 0 is 0.85x the uniform baseline, slightly below chance — so the massive sinks StreamingLLM exploits are an emergent property of scale. The listing demonstrates the measurement methodology while confirming the phenomenon is scale-dependent, which is itself the interview-relevant point about extrapolating small-scale intuitions.

Q24. How does quantization affect inference, and what is KV-cache quantization?

Because decoding is memory-bound (Q7), shrinking the bytes moved directly cuts latency and lets bigger batches/contexts fit. Weight-only quantization (INT8/INT4, via GPTQ/AWQ — Chapter 21) is the common case for memory-bound decode: smaller weights, faster load, near-free quality at INT8. Weight+activation INT8 helps compute-bound prefill via tensor cores. KV-cache quantization stores K/V in INT8 or FP8 to halve or quarter the fastest-growing memory consumer, enabling longer contexts and bigger batches at a small accuracy cost (K is more sensitive than V; per-channel/group scaling helps). The related architectural lever is GQA/MQA — sharing K/V across query heads to shrink the cache at the source.

Q25. Users complain the chatbot is slow — high time-to-first-token on long prompts and sluggish streaming. Diagnose and fix.

Separate the two metrics. High TTFT is a prefill problem: it scales with prompt length, so cache and reuse shared prefixes/system prompts (prefix caching), use chunked prefill so a long prompt does not monopolize the GPU, and shorten prompts (RAG top-k, compression). Sluggish streaming is an inter-token-latency problem in memory-bound decode: add speculative decoding (free latency, no quality cost), quantize weights and KV cache to cut bytes moved, use GQA/a smaller model or distillation if quality budget allows, and ensure continuous batching so decode slots stay full. If both are bad under load, the deeper cause is likely batching/memory: check KV utilization (PagedAttention) and whether prefill and decode are contending — disaggregating them is the heavier fix.

Q26. A downstream service needs guaranteed-valid JSON matching a schema. How do you deliver it, and what remains uncovered?

Use constrained decoding (Q19): compile the JSON schema into a grammar/FSM and mask logits each step so only schema-valid tokens can be emitted — this guarantees syntactic validity and correct types/enums by construction, unlike prompt-and-retry which fails a fraction of the time. Add a low temperature or greedy decoding for the structured fields to reduce spurious variation. What remains uncovered: semantic correctness — a schema-valid object can still contain a hallucinated or wrong value, so validate values against ground truth or tools, and keep field descriptions in the prompt so the model fills them sensibly. Also budget for the grammar occasionally forcing low-probability paths; test that the schema is not so tight it degrades content quality.

Q27. This chapter's inference results in one sweep.

Temperature dial: 0.01 bits at T=0.5 to 8.51 at T=2.0; top-p nucleus 13 tokens (1.7-bit context) vs 256 (7.9-bit), adaptive where top-k is not (Listing 1). Beam trap: beam-8 best likelihood -1.64 but repetition 0.46/distinct-2 0.64, vs sampling -3.99 likelihood but 0.10/0.99 (Listing 2). KV cache: exact to 5e-15, 2.8-3.5x speedup (Listing 3). Speculative decoding: unbiased (TV 0.079 vs 0.082 target baseline), accept 0.09 -> 0.30 as draft ppl 1466 -> 46 (Listing 4). PagedAttention: utilization 16% -> 98%, prefix sharing saves 70% (Listing 5). Continuous batching: GPU 21% -> 74%, throughput +249%, latency 1026 -> 35 steps (Listing 6). Hallucination: ECE 0.060 (every bin overconfident), abstention 0.11 -> 0.43 on the confident subset, constrained decoding invalid 0.275 -> 0 (Listing 7). Context: perplexity within 5% of full from ~8 recent tokens; attention sink absent at 200k params (0.85x uniform) — emergent at scale (Listing 8).

Q28. A colleague says "inference is just calling `model.generate()` — the interesting work is training." Rebut it.

Inference is where a trained distribution becomes a product, and every layer of it is a real engineering problem. The decoding algorithm alone changes output quality categorically — the same weights produce repetitive garbage under beam search and fluent text under top-p (Listings 1-2). The serving stack changes cost and latency by an order of magnitude: KV caching turns $O(T^2)$ into $O(T)$ exactly (Listing 3), speculative decoding halves latency for free (Listing 4), PagedAttention recovers 6x the usable memory (Listing 5), and continuous batching triples throughput (Listing 6) — none of that is `model.generate()`. And the two behaviors users judge the product on, hallucination and long-context handling, are inference-time concerns with their own diagnostics and mitigations — calibration, abstention, constrained decoding, retrieval, position interpolation (Listings 7-8). Training sets the ceiling; inference decides whether you reach it, how fast, and at what price.